

HASHTABLE

자료구조 스터디 Week12

학생회

 Made with Gamma

INDEX

01

THEORY

02

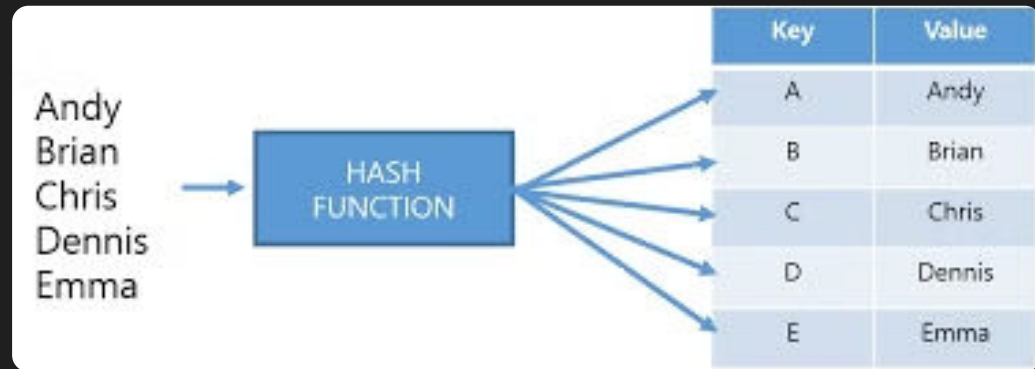
IMPLEMENTATION

03

PROBLEM SOLVING

해시테이블

해시테이블은 (key, value)로 데이터를 저장하는 자료구조 중 하나로 빠르게 데이터를 검색할 수 있습니다. 해시함수를 통해 임의의 key값을 한정된 정수로 매핑 시켜 배열의 인덱스로 사용할 수 있습니다.



해시함수

해시함수: key값을 하나의 정수로 대응시키는 함수

(Input - key, Output - index)

$a \leftarrow 0$

apple = 45

$b \leftarrow 1$

banana = 27

$c \leftarrow 2$

...

$z \leftarrow 25$

tomato = 79

apple = $45 \bmod 5 = 0$

banana = $27 \bmod 5 = 2$

tomato = $79 \bmod 5 = 4$

해시테이블

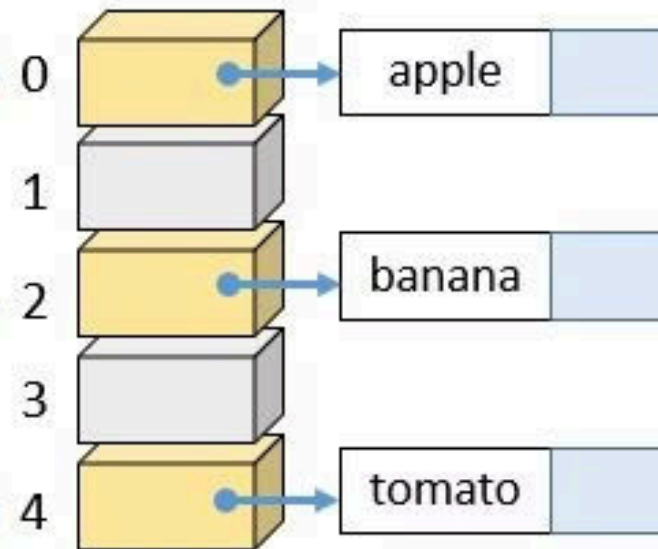
$h_2: \text{integer} \rightarrow [0, N-1]$

$h_2 = h_1(\text{key}) \bmod N$

apple = $45 \bmod 5 = 0$

banana = $27 \bmod 5 = 2$

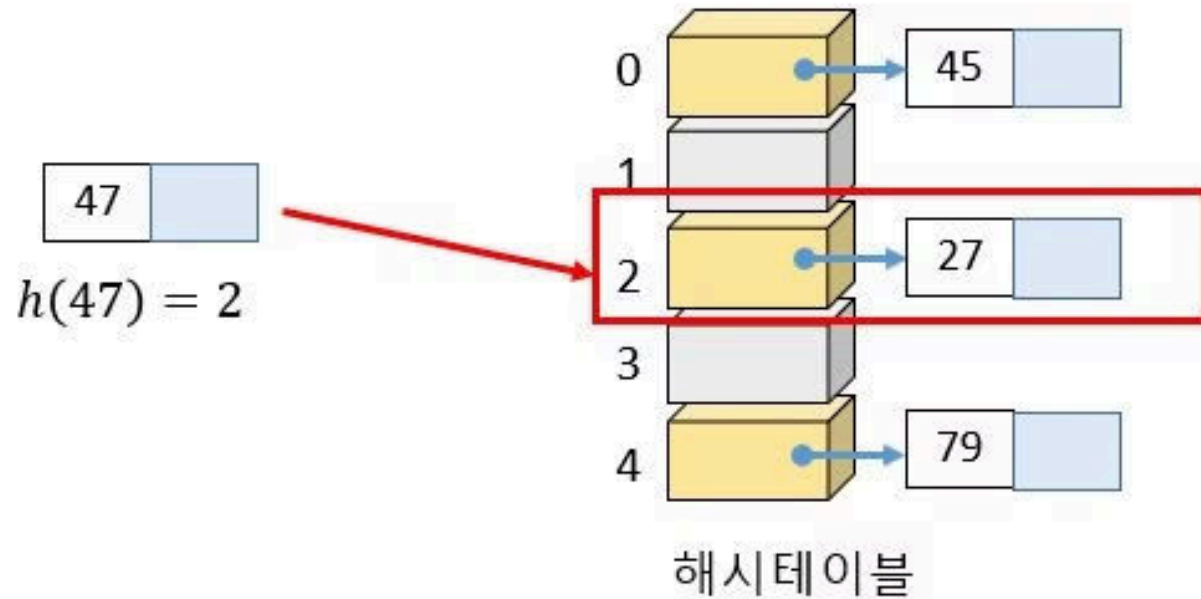
tomato = $79 \bmod 5 = 4$



해시테이블
(크기:5)

충돌(Collision)

충돌 : 서로 다른 두 개의 key k_1, k_2 에 대하여 $h(k_1) = h(k_2)$ 인 경우



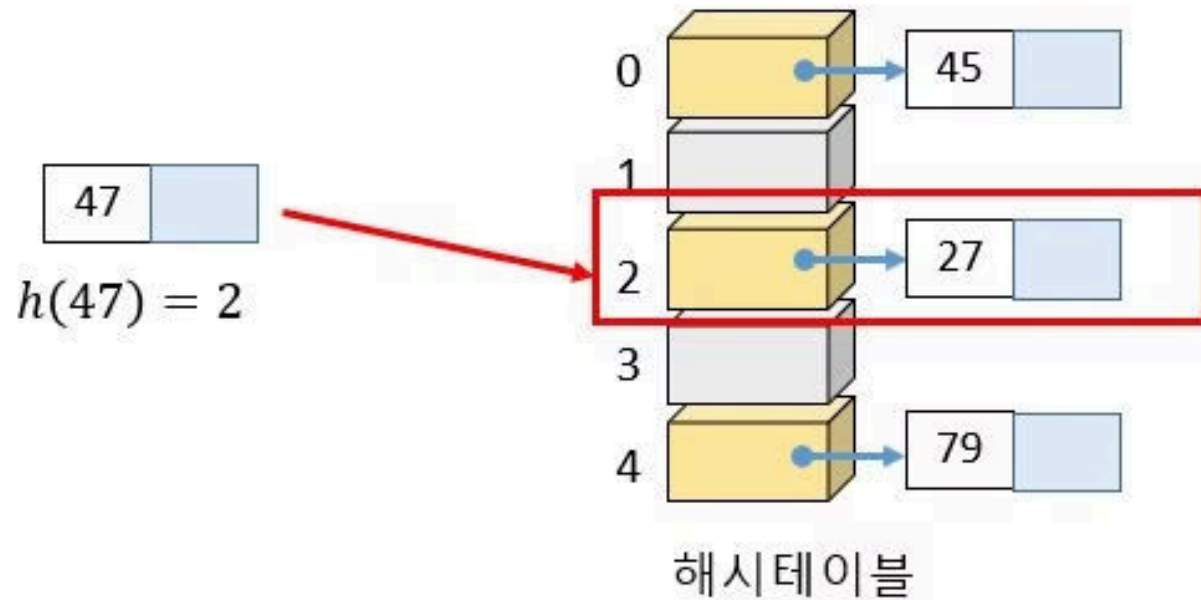
충돌 처리 기법

- 체이닝 - 충돌이 일어난 항목을 배열의 한 셀에 리스트 형태로 저장
- 오픈 어드레싱 - 충돌이 일어난 항목을 배열의 다른 셀에 저장
 - 선형 조사(Linear probing)
 - 이중 해싱(Double hashing)

LINEAR PROBING

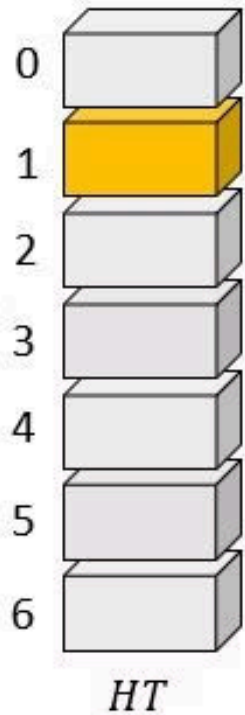
선형 조사

선형 조사 : 배열의 다음 인덱스가 비어있는지 확인



선형 조사 - 삽입

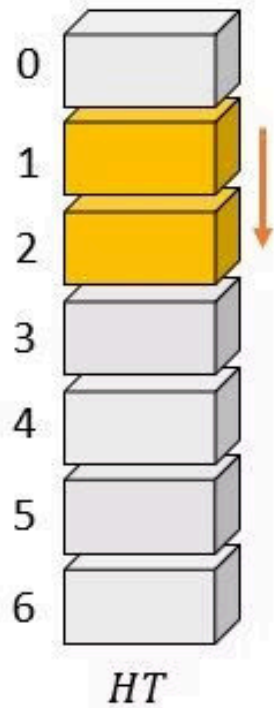
- $h(k) = k \bmod 7$ 일 때, (8, 1, 9, 6, 13)을 차례로 삽입



k	h(k)	Probes		
8	1	1		

선형 조사 - 삽입

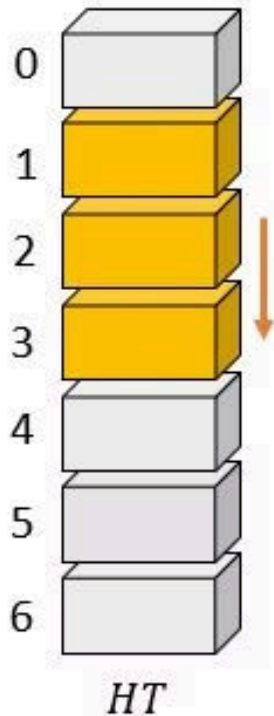
- $h(k) = k \bmod 7$ 일 때, (8, 1, 9, 6, 13)을 차례로 삽입



k	h(k)	Probes		
8	1	1		
1	1	1	2	

선형 조사 - 삽입

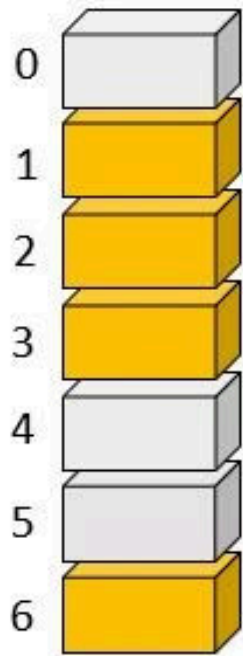
- $h(k) = k \bmod 7$ 일 때, (8, 1, 9, 6, 13)을 차례로 삽입



k	h(k)	Probes		
8	1	1		
1	1	1	2	
9	2	2	3	

선형 조사 - 삽입

- $h(k) = k \bmod 7$ 일 때, (8, 1, 9, 6, 13)을 차례로 삽입

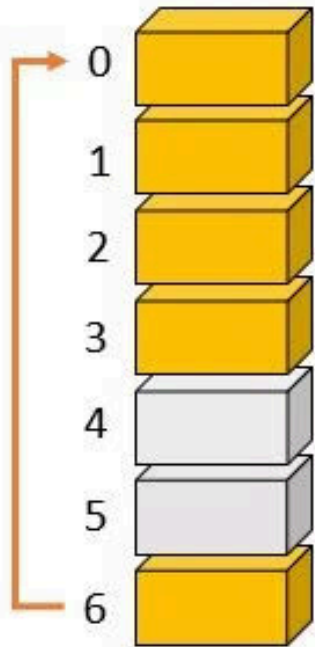


HT

k	h(k)	Probes		
8	1	1		
1	1	1	2	
9	2	2	3	
6	6	6		

선형 조사 - 삽입

- $h(k) = k \bmod 7$ 일 때, (8, 1, 9, 6, 13)을 차례로 삽입

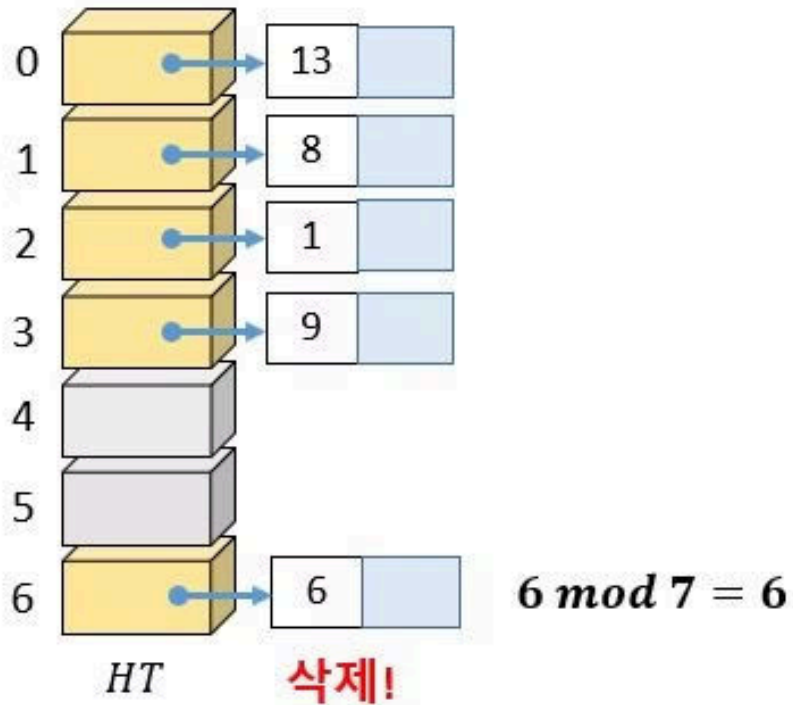


HT

k	h(k)	Probes		
8	1	1		
1	1	1	2	
9	2	2	3	
6	6	6		
13	6	6	0	

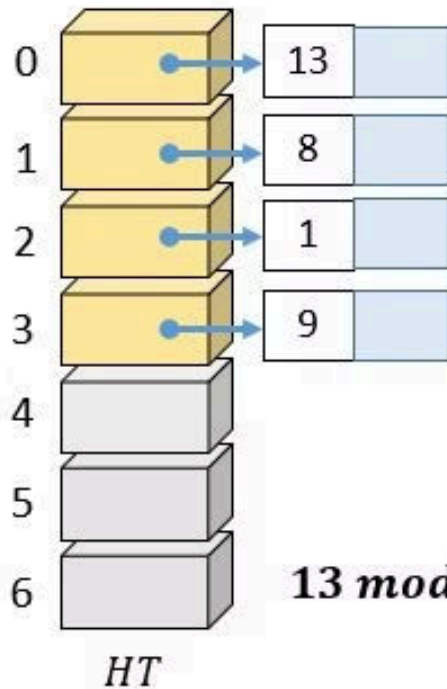
선형 조사 - 삭제

- $h(k) = k \bmod 7$ 일 때, (8, 1, 9, 6, 13) 삽입 후, (6, 13)을 차례로 삭제



선형 조사 - 삭제

- $h(k) = k \bmod 7$ 일 때, (8, 1, 9, 6, 13) 삽입 후, (6, 13)을 차례로 삭제

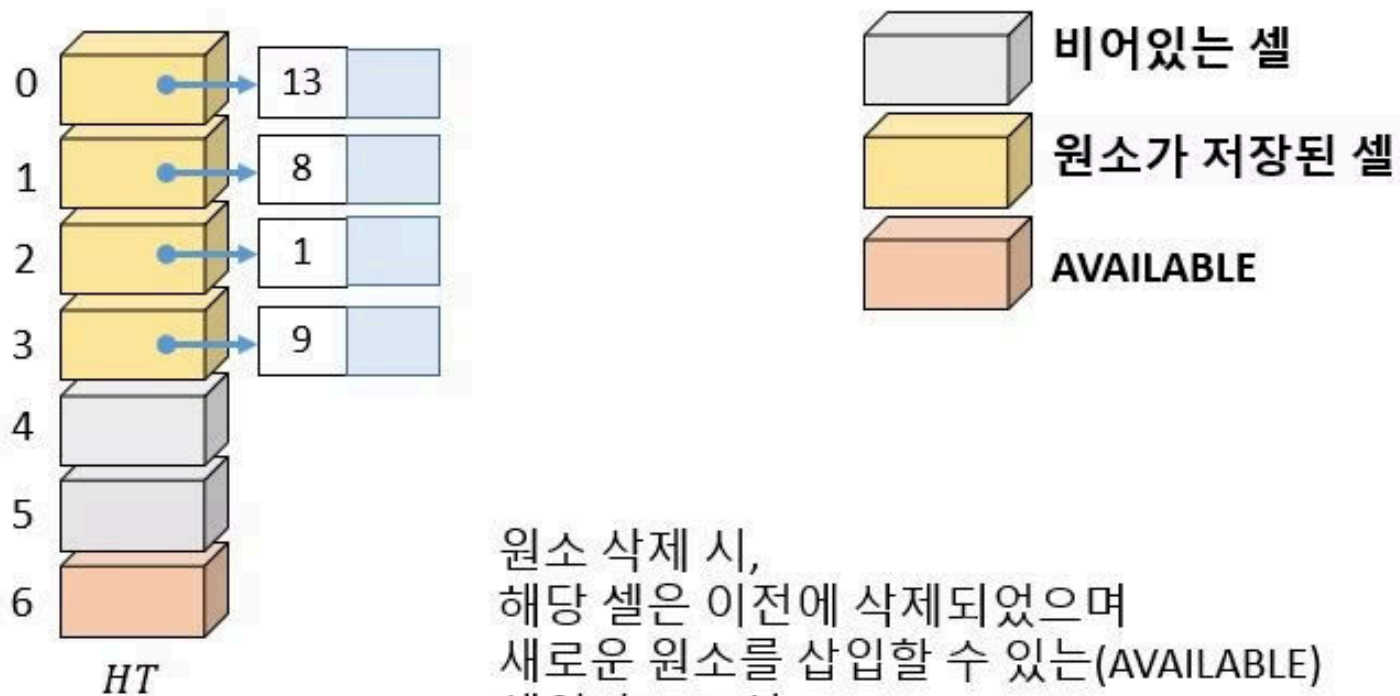


$$13 \bmod 7 = 6$$

$HT[0]$ 을 탐색해야 하지만
 $HT[6]$ 이 비어있는 상태이기 때문에
탐색하지 않고 종료됨

선형 조사 - 삭제

- $h(k) = k \bmod 7$ 일 때, (8, 1, 9, 6, 13) 삽입 후, (6, 13)을 차례로 삭제



원소 삭제 시,
해당 셀은 이전에 삭제되었으며
새로운 원소를 삽입할 수 있는(AVAILABLE)
셀이라고 표시

DOUBLE HASHING

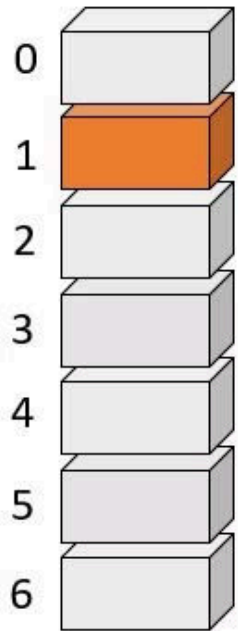
이중 해싱

이중 해싱 : 충돌이 발생하면 추가적인 해시 함수를 사용

- $i = h(k) = k \bmod N, \quad d(k) = q - (k \bmod q)$
- $i_j = (i + jd(k)) \bmod N \quad (i = 0, 1, \dots, N - 1)$
- N, q 는 소수
- $HT[i_0] \rightarrow HT[i_1] \rightarrow \dots \rightarrow HT[i_{(N-1)}]$

이중 해싱 - 삽입

- $h(k) = k \bmod 7$, $d(k) = 5 - (k \bmod 5)$ 일 때, (8, 1, 9, 6, 13)을 차례로 삽입

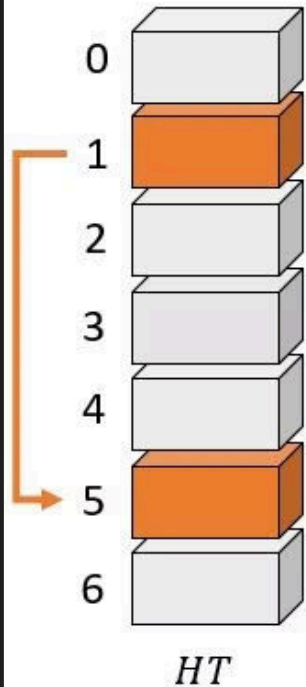


HT

k	h(k)	d(k)	Probes		
8	1	2	1		

이중 해싱 - 삽입

- $h(k) = k \bmod 7$, $d(k) = 5 - (k \bmod 5)$ 일 때, (8, 1, 9, 6, 13)을 차례로 삽입

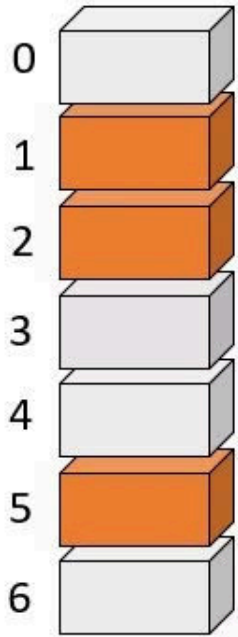


k	h(k)	d(k)	Probes		
8	1	2	1		
1	1	4	1	5	

$1+4 \Rightarrow 5 \bmod 7$

이중 해싱 - 삽입

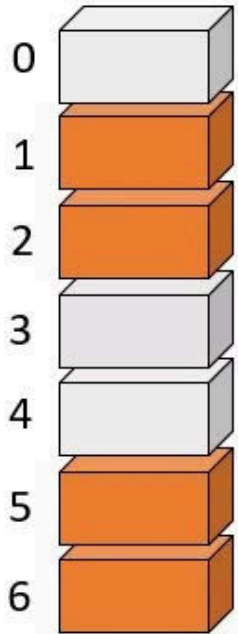
- $h(k) = k \bmod 7$, $d(k) = 5 - (k \bmod 5)$ 일 때, (8, 1, 9, 6, 13)을 차례로 삽입



k	h(k)	d(k)	Probes		
8	1	2	1		
1	1	4	1	5	
9	2	1	2		

이중 해싱 - 삽입

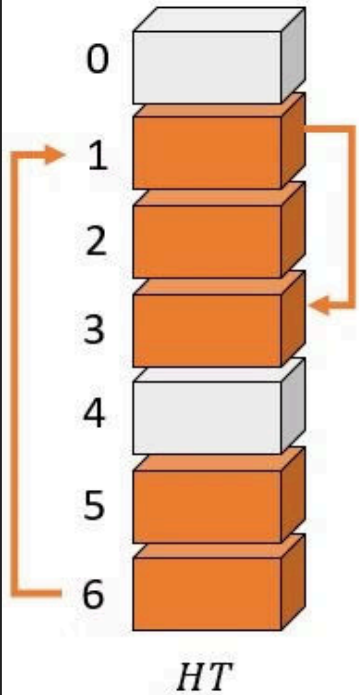
- $h(k) = k \bmod 7$, $d(k) = 5 - (k \bmod 5)$ 일 때, (8, 1, 9, 6, 13)을 차례로 삽입



k	h(k)	d(k)	Probes		
8	1	2	1		
1	1	4	1	5	
9	2	1	2		
6	6	4	6		

이중 해싱 - 삽입

- $h(k) = k \bmod 7$, $d(k) = 5 - (k \bmod 5)$ 일 때, (8, 1, 9, 6, 13)을 차례로 삽입



k	h(k)	d(k)	Probes		
8	1	2	1		
1	1	4	1	5	
9	2	1	2		
6	6	4	6		
13	6	2	6	1	3

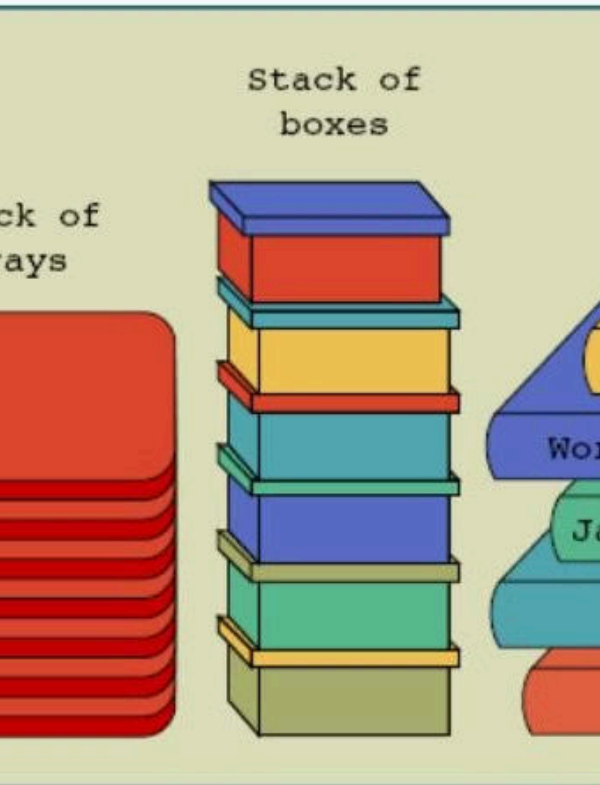
$$6 + 2 + 2 \Rightarrow 10 \bmod 7$$

결론

- Probe - 조사 횟수
- Entry - (key, value)
- 선형 조사와 이중 해싱은 인덱스를 찾는 방법(해시함수)만 상이
- 삽입 - 해당 인덱스에 엔트리 삽입, 충돌 발생 시 해시함수로 다음 인덱스 확인
- 삭제 - 해당 인덱스에 엔트리가 존재한다면 삭제, 존재하지 않는다면 해시함수로 다음 인덱스 확인
- 탐색 - 해당 인덱스에 엔트리가 존재한다면 리턴

IMPLEMENTATION

Stacks



Stacks of stacks

해시테이블의 구현 방법

1

Linear Probing

선형 조사

2

Double Hashing

이중 해싱

스택의 기본 연산

1 HashFnc

해시함수

3 Erase

엔트리 삭제

2 Put

엔트리 삽입

4 Find

엔트리 탐색

LINEAR PROBING

매크로

```
1  #include <iostream>
2  using namespace std;
3
4  #define NOITEM 0
5  #define ISITEM 1
6  #define AVAILABE 2
7
```

- NOITEM - 인덱스에 엔트리가 존재하지 않음
- ISITEM - 인덱스에 엔트리가 존재
- AVAILABE - 인덱스에 엔트리가 삭제된 적 있음

엔트리 구조체

```
8  ▼ struct entry {
9      int key;
10     string value;
11     int valid;
12
13  ▼ entry() {
14      key = 0;
15      value = "";
16      valid = NOITEM;
17  }
18
19  ▼ entry(int key, string value) {
20      this->key = key;
21      this->value = value;
22      valid = ISITEM;
23  }
24
25  void erase() {
26      valid = AVAILABLE;
27  }
28  };
```

- 기본 - NOITEM
- 생성 - ISITEM
- 삭제 - AVAILABLE

멤버변수

```
30  ✓  class HashTable {  
31      private:  
32          entry* table;  
33          int cap;  
34          int HashFnc(int key) {  
35              return key % cap;  
36          }
```

- table - 해시테이블(배열)
- cap - 해시테이블의 크기
- HashFnc - 해시함수

생성자

```
38     public:
39         HashTable(int N) {
40             cap = N;
41             table = new entry[cap];
42         }
43
```

- 인자로 받은 크기의 해시테이블 생성

삽입

```
44 void put(int key, string value) {  
45     int idx = HashFnc(key);  
46     int probe = 1;  
47  
48     while (table[idx].valid == ISITEM && probe <= cap) {  
49         idx = HashFnc(idx + 1);  
50         probe++;  
51     }  
52  
53     if (probe > cap)  
54         return;  
55  
56     cout << probe + idx << '\n';  
57     table[idx] = entry(key, value);  
58 }
```

- 해시함수로 idx 구하기
- 조사 횟수 1로 시작
- 해당 idx가 ISITEM(충돌)이라면 다음 idx 구하기
- idx가 결정 되었는데 조사 횟수가 해시테이블의 크기 보다 크다면(빈 공간 없음) 종료
- 해당 idx에 엔트리 삽입

삭제

```
60 void erase(int key) {
61     int idx = HashFnc(key);
62     int probe = 1;
63
64     while (table[idx].valid != NOITEM && probe <= cap) {
65         if (table[idx].valid == ISITEM && table[idx].key == key) {
66             cout << table[idx].value << '\n';
67             table[idx].erase();
68             return;
69         }
70
71         idx = HashFnc(idx + 1);
72         probe++;
73     }
74
75     cout << "None\n";
76 }
```

- 해시함수로 idx 구하기
- 조사 횟수 1로 시작
- 해당 idx가 NOITEM이 아닌 경우에(ISITEM or AVAILABLE), 해당 idx가 ISITEM이고, key값이 일치한다면 삭제할 인덱스 탐색에 성공 → 삭제
- 해당 idx가 NOITEM이 아닌 경우에 위 조건에 부합하지 않는다면 다음 idx 구하기

탐색

```
78  ✓    string find(int key) {  
79          int idx = HashFnc(key);  
80          int probe = 1;  
81  
82          while (table[idx].valid != NOITEM && probe <= cap) {  
83              if (table[idx].valid == ISITEM && table[idx].key == key) {  
84                  return table[idx].value;  
85              }  
86  
87              idx = HashFnc(idx + 1);  
88              probe++;  
89          }  
90  
91          return "None";  
92      }
```

- 삭제와 거의 유사(삭제의 전제가 탐색이기 때문)
- 조건 만족 시 삭제 대신 리턴해주면 됨

DOUBLE HASHING

선형 조사와의 차이점

해시함수 및 해시함수를 이용하여 idx를 구하는 코드만 수정 (//)

```
30 class HashTable {
31     private:
32         entry* table;
33         int cap;
34         int div; //
35         int HashFnc(int key) {
36             return key % cap;
37         }
38         int HashFnc2(int key) { //
39             return div - (key % div);
40         }
41     public:
42     HashTable(int N, int M) { //
43         cap = N;
44         div = M; //
45         table = new entry[cap];
46     }
47 }
```

```
49 void put(int key, string value) {
50     int idx = HashFnc(key);
51     int probe = 1;
52
53     while (table[idx].valid == ISITEM && probe <= cap) {
54         idx = HashFnc(idx + HashFnc2(key)); //
55         probe++;
56     }
57
58     if (probe > cap)
59         return;
60
61     cout << probe + idx << '\n';
62     table[idx] = entry(key, value);
63 }
```

선형 조사와의 차이점

해시함수 및 해시함수를 이용하여 idx를 구하는 코드만 수정

```
65 void erase(int key) {
66     int idx = HashFnc(key);
67     int probe = 1;
68
69     while (table[idx].valid != NOITEM && probe <= cap) {
70         if (table[idx].valid == ISITEM && table[idx].key == key) {
71             cout << table[idx].value << '\n';
72             table[idx].erase();
73             return;
74         }
75
76         idx = HashFnc(idx + HashFnc2(key)); //
77         probe++;
78     }
79
80     cout << "None\n";
81 }
```

```
83 string find(int key) {
84     int idx = HashFnc(key);
85     int probe = 1;
86
87     while (table[idx].valid != NOITEM && probe <= cap) {
88         if (table[idx].valid == ISITEM && table[idx].key == key) {
89             return table[idx].value;
90         }
91
92         idx = HashFnc(idx + HashFnc2(key)); //
93         probe++;
94     }
95
96     return "None";
97 }
```

PROBLEM SOLVING

**THANK'S FOR
WATCHING**